

What is the Latency?

It is the time spent for **executing a single task in a computation**.

What is the Completion Time?

The time spent **since the beginning** of the execution of the **first** task **through the end** of the **last** task.

What is the Throughput?

The amount of tasks computed per unit of time

What is the Service Time?

Service time refers to the **duration required to produce the results of a task within a sequence, starting immediately after the results of the preceding task have been delivered.**

What is Data Parallelism?

Data Parallelism is achieved by **dividing an initial task into subtasks, whose partial results are then combined to produce the final outcome**. The effort required to **partition the task** and **merge the partial results** constitutes the **computational overhead**. Different Data Parallelism techniques vary in how they generate subtasks and aggregate results. In **embarrassingly parallel** computations, the task is easily divided into **completely independent subtasks**, with a straight-forward merging of their results. The main challenge of this approach lies in balancing the **time spent splitting tasks and managing parallel execution against the performance gains from parallelism**.

What is Stream Parallelism?

Stream Parallelism involves the **parallel execution of tasks from an input stream, where tasks become available at different times**. Unlike other forms of parallelism, **not all tasks are available simultaneously**. The **interarrival time** between tasks **significantly impacts** the potential for parallel computation. **If there are no dependencies between tasks, the computation becomes embarrassingly parallel**. Stream Parallelism **doesn't reduce the execution time of individual tasks** but **increases the overall throughput of the application**, which in turn **reduces the total completion time**. It's important to note that the stream of tasks can be infinite.

What is the difference between Data Parallelism and Stream Parallelism?

Data Parallelism reduces the **execution time** (latency) of a **task by breaking it down into smaller subtasks, each processing a portion of the input data in parallel**. In contrast, **Stream Parallelism** does not reduce the latency of individual tasks. Instead, it **improves the overall throughput of the application by executing multiple independent tasks concurrently when available**, thereby reducing the total completion time of the application.

What is Control Parallelism?

Control Parallelism occurs when **different parts of the same control flow are executed in parallel**. For instance, in an **if-then-else** construct, the computation of the guard, the **"then"** branch, and the **"else"** branch can all be executed **concurrently**. Once the guard is evaluated, the correct branch is chosen, and the task computing the other branch is discarded, even if it hasn't finished. **This allows the execution of the chosen branch to either be completed or nearly finished by the time the guard is resolved, improving overall efficiency.**

What is Task Parallelism?

Task Parallelism involves breaking down a computation into multiple tasks, which may have dependencies. Tasks without dependencies can be executed in parallel. A task processes input data and produces output. Two tasks, x_i and x_j , are dependent if one cannot begin until the other has finished because one task's output is required as input for the other. Task Parallelism helps reduce the overall latency of a computation by allowing independent tasks to be processed concurrently.

How can we represent Task Parallelism?

In Task Parallelism a computation is modelled as a graph where each node is a task and each edge represent a dependency between two tasks. The initial node as no dependencies.

How to model Data Parallel computations using Task Parallelism?

A Data Parallel application can be modeled by a task that splits the input data into partitions. Each partition is then assigned to a set of tasks, with one task responsible for collecting and merging the results to reconstruct the final output. Dependencies exist between the initial task that performs the data split, the tasks processing each partition, and the task responsible for assembling the final result.

How to model Stream Parallel computations using Task Parallelism?

We model it by creating a task for each one of the input data sets to compute.

The concept of Structured Programming and Unstructured Programming is the same between Sequential Programs and Parallel Programs?

No, the concepts differ significantly between the two.

- In **Sequential Programs**, **Structured** Programming refers to programming languages where control flow is organized using well-defined constructs such as conditionals, loops, and function calls. **Unstructured** Programming, on the other hand, involves transferring control via GOTO statements, which can lead to less organized and harder-to-maintain code.
- In **Parallel Programs**, **Structured** Programming refers to models where parallelism is achieved using well-established patterns that represent common parallel computations (e.g. the building blocks in FastFlow or constructs in OMP). In contrast, **Unstructured** Programming relies on lower-level abstractions like threads, processes, and communication or synchronization primitives, requiring the programmer to manage parallel execution manually (e.g. the std::thread library).

What are the advantages of Structured Parallel Programs compared to Unstructured Ones?

Structured Parallel Programs offer several benefits: they are easier to design and implement, as many parallelism-related responsibilities are shifted from the programmer to the tools. This allows for the possibility of automatic tuning and optimization. Additionally, structured parallel programs provide better portability, ensuring consistent functionality and performance across different systems to a certain extent.

What are "shapes" in Parallel Programming?

In parallel programming a shape is a **set of parallel computations with dependencies that take place and require proper orchestration to contribute on the delivery of the final result**. Shapes are divided in Stream Parallel Shapes and Data Parallel Shape

What are the Stream Parallel Shape?

Stream Parallel Shapes are used to process **streams of items, transforming them into another stream of items**. Formally, a stream is defined as $\langle x_1, x_2, \dots, x_n, x_{n+1}, \dots \rangle$, where each item x_i becomes available at different points in time. Streams can also be **infinite**, continuously producing items over time.

The Stream Parallel Shapes are:

- **Pipeline**: **computes in parallel the composition of a number of functions $f_1, \dots, f_k$ over all the items of the streams**. Suppose α is the type of the items of the input stream, then the pipeline can be interpreted as:

$$\text{pipe}(f, g)(\langle x_1, x_2, \dots, x_n, x_{n+1}, \dots \rangle) = \langle f_k(f_{k-1}(\dots(f_1(x_1))\dots)), f_k(f_{k-1}(\dots(f_1(x_2))\dots)) \rangle$$

$$\text{pipe} : (\alpha \rightarrow \beta) \times (\beta \rightarrow \gamma) \times \langle \alpha \rangle \rightarrow \langle \gamma \rangle$$

Parallelism in pipeline is exploited in the **computation of different stages** (interpreted as functions) over partial results relative to different input streams.

- **Farm**: applies the **same function f to all the items of an input stream**. A farm can be interpreted as the parallel computation of:

$$\text{farm}(f)(\langle x_1, x_2, \dots \rangle) = \langle f(x_1), f(x_2), \dots \rangle$$

$$\text{farm} : (\alpha \rightarrow \beta) \times \langle \alpha \rangle \rightarrow \langle \beta \rangle$$

In general the **input-output ordering may not be required**, that is items of the output stream can be delivered out of order. **Farms are embarrassingly parallel $\forall i \in [1, \dots]. f(x_i)$ can be computed in parallel**.

What are the Data Parallel Shape?

All Data Parallel Shapes operate on an **input "data collection"** to produce a result, typically derived from the results of subcomputations performed on partitions of the data. It is possible for the input collection to have **overlapping elements**.

The Data Parallel Shapes are:

- **Map**: This operation **applies a function f to each element of a collection $\mathcal{C}$** (which could be a vector, list, tree, etc.). **The result of $\text{map}(f, \mathcal{C})$ is a collection isomorphic to the original, such that for every $x \in \mathcal{C}$, there exists $f(x) \in \text{map}(f, \mathcal{C})$:**

$$\text{map}(f, \langle x_1, \dots, x_n \rangle) = \langle f(x_1), \dots, f(x_n) \rangle$$

where $f : \alpha \rightarrow \beta$ and $\text{map} : (\alpha \rightarrow \beta) \times \langle \alpha \rangle \rightarrow \langle \beta \rangle$.

Parallelism can be leveraged during the computation of $f(x_i)$ and $f(x_j)$ for any $i \neq j$. If the collection $\mathcal{C}$ contains m elements and there are m available threads, then **each $f(x_i)$, where $i = 1, \dots, m$, can be computed in parallel**. Consequently, the map operation is often referred to as **"embarrassingly parallel"**.

- **Reduce**: This operation aggregates a collection of data items using a typically associative and commutative binary function (such as $\oplus$, $\otimes$, etc.). The result of the computation is a single value, effectively reducing the collection to a scalar. When applying the binary operation $\oplus$, the reduction can be written as:

$$reduce(\oplus, \langle x_1, \dots, x_m \rangle) = x_1 \oplus x_2 \oplus \dots \oplus x_m$$

where $\oplus : \alpha \times \alpha \rightarrow \alpha$, and the reduce operation is defined as $reduce : (\alpha \times \alpha \rightarrow \alpha) \times \langle \alpha \rangle \rightarrow \alpha$. Parallelism in a reduce operation can be exploited, provided that the binary function is associative and commutative. The computation can be structured as a tree, where the leaves compute the operation on pairs of consecutive elements from the collection, and internal nodes compute the results of their left and right subtrees, continuing up to the root node.

- **Prefix**: computes the vector of the m partial "sums" of a collection with m items, using a usually associative and commutative binary function (such as $\oplus$, $\otimes$, etc.):

$$prefix(\oplus, \langle x_1, \dots, x_n \rangle) = \langle x_1, x_1 \oplus x_2, x_1 \oplus x_2 \oplus x_3, \dots \rangle$$

where $prefix : (\alpha \times \alpha \rightarrow \alpha) \times \langle \alpha \rangle \rightarrow \alpha$

- **Stencil**: this operation takes two functions, f and g , and applies them to a collection of items, producing an isomorphic output collection. The i^{th} item in the result is computed in parallel as follows:

$$stencil(f, g, \langle x_1, \dots, x_m \rangle) = \langle f(g(1, \langle x_1, \dots, x_m \rangle)), \dots, f(g(m, \langle x_1, \dots, x_m \rangle)) \rangle$$

where $g(i, \langle x_1, \dots, x_m \rangle)$ computes the neighborhood set for the i^{th} item in the input collection, and f applies a function to the elements of this neighborhood. The stencil operation is defined as $stencil : (\langle \alpha \rangle \rightarrow \beta) \times (item \times \langle \alpha \rangle) \times \langle \alpha \rangle \rightarrow \langle \beta \rangle$. Parallelism in a stencil operation can be exploited similarly to map patterns. However, when replacing the input collection with the resulting output, care must be taken to ensure that neighborhood computations access the original (input) values rather than newly computed ones.

- **Divide & Conquer**: it computes a result by dividing the input task in subtasks, computing per each subtask a partial result which will be combined into the final result. We define the function as $divide \ \& \ conquer(isBc, solveBc, Divide, Conquer)$ s.t.:

- $isBc : \langle \alpha \rangle \rightarrow Bool$
- $solveBc : \langle \alpha \rangle \rightarrow \beta$
- $Divide : \langle \alpha \rangle \rightarrow \langle \langle \alpha \rangle \rangle$
- $Conquer : \langle \beta \rangle \rightarrow \beta$

where if $isBc(\langle x_1, \dots, x_m \rangle)$ holds true, then $solveBc(\langle x_1, \dots, x_m \rangle)$ will be returned, otherwise return:

$$Conquer(map(divide \ \& \ conquer(isBc, Bc, Divide, Conquer), Divide(\langle x_1, \dots, x_m \rangle))$$

where $divide \ \& \ conquer : (\langle \alpha \rangle \rightarrow Bool) \times (\langle \alpha \rangle \rightarrow \beta) \times (\langle \alpha \rangle \rightarrow \langle \langle \alpha \rangle \rangle) \times (\langle \beta \rangle \rightarrow \beta) \rightarrow \beta$. Parallelism in a $divide \ \& \ conquer$ may be exploited in the map of $divide \ \& \ conquer$ calls over the results of the $Divide$ operation

Difference between Map and Farm

Map and Farm are both parallel shapes, but they differ in structure and application.

Map is a **data-parallel shape**, where a function f is applied to each element of a given collection $\mathcal{C}$. The result, $map(f, \mathcal{C})$, is a new collection $\mathcal{C}'$ that is isomorphic to the original. The **parallelism** in the Map pattern arises from **computing the function on distinct elements of the collection concurrently**.

The **Farm** pattern is a **stream-parallel shape**. It applies a function f to all items in an input stream. The output does **not need to preserve the ordering or structure of the input collection**. Farms can be **parallelized when the processing of distinct elements is independent**, allowing for concurrent execution of tasks without dependencies between them.

How to choose the number of Workers in a Farm by using the Service Time?

Recall that the Service Time is defined as the duration required to produce the results of a task within a sequence, starting immediately after the results of the preceding task have been delivered.

There are two equivalent answers to this question:

- By interpreting the Farm as a **three-stage pipeline**, its **Service Time** depends of the three actors in it:
 - **Emitter**: will have to **retrieve the task from the input stream, eventually split it into subtasks, and send it to the Worker(s)**. We denote its Service Time as t_E
 - **Worker(s)**: assuming we have n_w Workers, then they compute n_w tasks and **send them to the Collector concurrently**. The time that each Collector spends for computing a single task is in t_W time.
 - **Collector**: **retrieves the tasks from the Workers and manage them**. We denote its Service Time as t_C

Thus the Service Time of the Farm is the maximum between these three actors:

$$Service_Time(Farm) = \max(t_E, \frac{t_W}{n_w}, t_C)$$

Note that **the only term that depends on the number of Workers n_w is $\frac{t_W}{n_w}$** . Thus the only way to optimize the Service Time using the number of workers is to made the dependent term not the maximum, meaning:

$$\frac{t_W}{n_W} \leq \max(t_E, t_C) \Rightarrow n_W > \lceil \frac{t_W}{\max(t_E, t_C)} \rceil$$

- Here we interpret the **Farm as a Client-Server setup** where the **Emitter and Collector** are **clients**, while the **Workers are servers**. Recall that in a Client-Server setting we have the following measures:
 - **Interarrival Time**: The Interarrival Time is the **time between the arrivals of two consecutive clients or requests at the server**. In other words, it measures how **frequently** new requests **(clients) arrive**. Formally:
$$Interarrival_Time = t_2 - t_1$$
 - **Departure Time**: The Departure Time is the **time at which a client or request finishes receiving service from the server and leaves the system**. In other words, it is the time **when the server has completed processing the request** and the client has departed.

- **Utilization Factor**: The utilization factor in a client-server setting, particularly for an input queue, is a measure of how much of the server's capacity is being used to process client requests. It helps determine whether the server is under-utilized, optimally utilized, or overloaded.

Now considering our Farm setting, the **Emitter** send tasks to the Workers, thus has an **Interarrival Time** that we declare as $t_{E,A}$. Since we have **only one Emitter**, its execution is **sequential** and its **Departure Time** is $t_{E,P} = t_{E,A}$. Assuming we are using a **Round Robin Scheduling** (like FastFlow whoooooo! (awcutawly Fast Flow uses loose Round Robin *nerdemoji*)) for sending the tasks to the n_w Workers, the probability that Worker W_i receives a task is:

$$p_i = \frac{1}{n_w}$$

The **Interarrival time of a Worker** becomes:

$$t_A = \frac{t_{E,A}}{\frac{1}{n_w}} = n_w \times t_{E,A}$$

The **Utilization Factor of the input queue** becomes:

$$p = \frac{t_w}{n_w \times t_{E,A}}$$

In order to have the Worker satisfy its Service Time T_W , the **Utilization Factor** should be **smaller than 1**, which means:

$$p = \frac{t_w}{n_w \times t_{E,A}} < 1 \Rightarrow t_w < n_w \times t_{E,A} \Rightarrow n_w > \frac{t_w}{t_{E,A}}$$

What happens if I implement a Map using a Farm

The key difference between a Map and a Farm structure lies in their **type shape**. A Map is a **data-based** shape, meaning the **entire collection of tasks is known and available at the beginning of execution**. In contrast, a Farm is a stream-based shape, continuously receiving tasks throughout execution s.t. **there is never a moment when all tasks are queued upfront**.

In a Map, knowing the total number of tasks at the outset **allows us to determine at the start the number of Workers needed to evenly distribute the workload across them**. However, if we implement a Map as though it were a Farm, **we wouldn't assume the entire collection is available from the start**. As a result, we would **need to determine the number of workers based on other factors, such as the interarrival time of tasks** (previous question). **This approach could lead to using too many or too few workers**, causing **imbalances** in task distribution.

Which techniques for distributing the workload have we seen?

We have seen **static** distribution techniques, where the **task assignment to threads is predetermined at the beginning of the program**. In this approaches, the **workload** is carefully **analyzed in advance**, allowing us to **select an optimal thread distribution** before execution begins.

The general idea assumes that we have m tasks, which significantly **exceed the available concurrent resources on the machine**. Creating m threads would introduce **inefficiencies** due to the **costly context switching involved in managing more threads than there are available resources**, as well as the **overhead** associated with repeatedly creating and destroying threads. Instead, we create p threads at the start (with p chosen based on the target architecture), **and each thread processes a block of tasks rather than a single task**.

We define the **Chunk Size** as the number of tasks each thread must process, where $c = \lceil \frac{m}{p} \rceil$. Note that each thread will handle at least $\lfloor \frac{m}{p} \rfloor$ tasks.

The techniques focus on **determining which tasks are assigned to each thread**.

- **Static Block Distribution:** We assign c consecutive tasks to each thread. If we consider the tasks as rows in a matrix, this means each thread is responsible for a contiguous block of rows. This approach is advantageous for exploiting cache locality, as it ensures that the data being processed by each thread is stored close together in memory.

Number of task	0	1	2	3	4	5	6	7	8
Number of thread	0	0	0	1	1	1	2	2	2

Figure 1: Block Distribution with $p = 3$ threads, $m = 9$ tasks, $c = \frac{9}{3} = 3$ chunks

- **Cyclic Distribution:** We assign c tasks to each of the p threads using a Round Robin distribution with a stride of p . This means tasks are distributed cyclically among the threads, ensuring that each thread receives a block of consecutive tasks in a repeating sequence.

Number of task	0	1	2	3	4	5	6	7	8
Number of thread	0	1	2	0	1	2	0	1	2

Figure 2: Cyclic Distribution with $p = 3$ threads, $m = 9$ tasks, $c = \frac{9}{3} = 3$ chunks

- **Block-Cyclic Distribution:** assume we have m tasks to process. We start by assigning $s = p \times c$ tasks to be handled in parallel by p threads. Each thread is given a fixed block of c tasks. If there are still tasks remaining ($m > s$), we continue the process, distributing the next set of tasks using a stride of s . Essentially, this approach implements a Cyclic Distribution of Fixed-Length task Blocks.

The parameter c can be used to adjust the desired properties of your distribution:

- **Small values c :** can be used in case of heterogeneous tasks that have to be implicitly load-balanced. However if c is too small then the resulting memory access patterns might exhibit suboptimal caching behaviour or even False Sharing.
- **High values c :** could lead to load-imbalance for heterogeneous tasks, but correspond to cache-friendly access patterns.

A reasonable trade-off is the following: we choose c to cover a complete cache line ($c = 16$ for 32-bit datatypes; $c = 8$ for 64bit datatypes). As a result we can guarantee cache-friendly access patterns while providing a high granularity of parallelism in order to avoid potential load-imbalance.

Number of task	0	1	2	3	4	5	6	7
Number of thread	0	0	1	1	0	0	1	1

Figure 3: Block-Cyclic Distribution with $p = 2$ threads, $m = 8$ tasks, $c = 2$ chunks, $s = 2 \times 2 = 4$ stride

What is the Cache Coherency Protocol?

A cache coherency protocol is a set of rules and mechanisms used in multicore and multiprocessors systems to ensure that multiple caches maintain a consistent view of shared memory. In a multi-core environment each core has its own local cache (i.e. L1 Cache). Local cache

can contain cache entries that refers to items presents in cache entries of other local caches. In order to avoid potential discrepancies, like if one processor updates its cache without informing the others, then the **cache coherency protocol ensures that any change made to a cached value by one processor is propagated to other caches that share the same item.** A common protocol include **MESI** (Modified, Exclusive, Shared, Invalid). **A issue that can arise cache coherency protocols is False Sharing.**

What is False Sharing?

A cache coherence protocol typically work on **cache line level**, i.e. if a core modifies a single value, all associated cache lines will be invalidated as a whole. **Assume a situation where several cores simultaneously operate on a distinct data items that are stored within the same main memory address region, that is are stored in a single cache line.** Now any write operation on that cache line would invalidate it in all cores that uses it. This means that **each core**, although it needs to read its own non-modified value, **is required to update the cache line by reading data from main memory.** This can lead to **severe performance degradation.**

As an example consider the following code executed concurrently for computing c chunk rows in a Matrix Vector Multiplication using Cyclic Distribution:

Algorithm 1 Threads computing concurrently a Matrix Vector Multiplication (False Sharing)

```

for  $row = id; row < c; row += num\_threads$  do
     $b[row] = 0$ 
    for  $col = 0; col < n; col ++$  do
         $b[row] += mtx[row][col] \times vec[col]$ 
    end for
end for

```

Although this Cyclic Scheme is easier to implement respect Block Distribution and performs at the same speed, it can lead to False Sharing.

The following algorithm is executed concurrently by p threads, each processing its own **chunk c of rows**. The **resulting vector** from the multiplication is denoted as b , with each thread storing its computed result for each row in $b[row]$. **Each thread uses its own local cache, and while threads theoretically access different elements of b , a single cache line might also include adjacent elements due to the principle of *locality of reference*.**

Although this approach avoids errors by ensuring that threads only access their assigned cells, it can lead to False Sharing. In the inner loop, threads update their local computations directly in b . While this might seem efficient at first, it results in **unnecessary cache line invalidations each time $b[row]$ is updated.** The cache coherence protocol forces all threads to update the cache line associated with b if $b[row]$ is present in their associated cache entry, even if that element is not used by other threads. Since this **update occurs every time $b[row]$ is written to** (which happens col times), **it can significantly impact the overall execution time.**

A better solution is the following:

Algorithm 2 Correct way to implement Matrix Vector Multiplication concurrently

```

for  $row = id; row < c; row += num\_threads$  do
     $res = 0$ 
    for  $col = 0; col < n; col ++$  do
         $res += mtx[row][col] \times vec[col]$ 
    end for
     $b[row] = res$ 
end for

```

Here we store the **local computations in a local variable res** , in this way we will **update b (and the common cache entry) only when we have actually provided the final result for that cell.**

A possible solution employed by Cache Coherency Protocols to False Sharing?

To avoid False Sharing there is a **solution employed by the Cache Coherency Protocol**. The solution consists in **padding a cache entry whose items are shared across multiple cores s.t. each cores in its cache entry contains only the interested item**. Another solution can be employed by the user by **programming the threads s.t. they make few access as possible to the local item in the common cache entry**. To do so local computation can be stored in a **temporary variable** and in the item we store only the final computation.

What states the **Amdahl's Law**?

The Amdahl's Law states that the **amount of non-parallelizable work in an application determines the maximum speedup we may achieve in parallelizing the application**. Given an application where we assumed that **s percent of the total work is sequential**, then we have **$p = (1 - s)$ percent of total works that can be parallelized**. Assume the whole **application** sequentially completes in **T_s unit of time**. Thus:

- **$s \times T_s$ is the part of the overall completion time that will remain sequential**
- **$(1 - s) \times T_s$ is the part of the overall completion time that can be parallelized**

Assume we have **n processing elements** (i.e. processes/threads), then the **execution time of the parallelizable part becomes:**

$$\frac{(1 - s) \times T_s}{n}$$

The overall **speedup** becomes:

$$speedup = \frac{T_s}{(s \times T_s) + ((1 - s) \times \frac{T_s}{n})}$$

By computing the limit for **$n \rightarrow \infty$** we can get the theoretic **limit** of the speedup for Amdahl's:

$$\lim_{n \rightarrow \infty} speedup = \lim_{n \rightarrow \infty} \frac{T_s}{(s \times T_s) + ((1 - s) \times \frac{T_s}{n})} = \frac{T_s}{s \times T_s} = \frac{1}{s}$$

What states the **Gustafsson's Law**?

After Gustafsson and Baris observed several applications that achieved a speedup far beyond what was expected by Amdahl's Law, they formulated a different speedup law.

Given s and p respectively the serial and parallel time spent on a parallel system with n computing elements, the speedup given is:

$$speedup(n) = \frac{s + (p \times n)}{s + p} = \frac{(s + p) + (n - 1) \times p}{s + p} = 1 + \frac{(n - 1) \times p}{s + p}$$

Then:

- the **scaled serial fraction f becomes:** $f = \frac{t_s}{t_s + t_p(n)}$
- the **scaled parallel fraction $(1 - f)$ becomes:** $(1 - f) = \frac{t_p(n)}{t_s + t_p(n)}$

The times for the parallel application are:

- **Parallel Time:** $T(n) = f + (1 - f) = 1$
- **Sequential Time:** $T(1) = f + (1 - f) \times n$
- **Scaled Speedup:** $\frac{T(1)}{T(n)} = \frac{f + (1-f) \times n}{1} = f + n + n \times f = n - (n - 1) \times f$

Gustafsson's Law is more suitable for modeling computations where the workload increases with the degree of parallelism, meaning the data size grows as more parallel resources are added. Unlike Amdahl's Law, which assumes a fixed problem size, Gustafsson's Law accounts for the fact that as we scale up resources, the computational workload can grow, making the overhead of managing parallelism relatively insignificant compared to the increasing data size. This allows for a more optimistic view of parallel scalability in scenarios where problem size expands with available resources.

What are the limitations of Amdahl's Law?

The Amdahl's Laws functions only in situations where the problem size is constant and the number of processors varies (strong scalability). When dealing with cases where the data size grows as more parallel resources are added (weak scalability), the time spent in the parallelizable part $(1 - f) \times T_s$ may grow faster in comparison to the non-parallelized part $f \times T_s$. For this reason Gustafsson introduced its law and the concept of Scaled Speedup.

What is the Scaled Speedup?

It is a generalization of the Gustafsson's Law (see the above question for a more complete answer):

$$\frac{T(1)}{T(n)} = \frac{p + (1 - p) \times n}{1} = p + n + n \times p = n - (n - 1) \times p$$

What guarantees enforced ordering of read and write operations?

In C++, it is considered undefined behavior to perform non-atomic read and write operations on the same data without enforcing a proper ordering. This ordering is ensured through operations on atomic variables, such as data-ready. Importantly, atomic operations not only enforce ordering on themselves but also apply this ordering to all preceding non-atomic operations. The memory model specifies this ordering through the relationships "happens-before" and "synchronized-with," which are established by atomic operations.

Why a Memory Model matters in parallel programming?

A memory model enables the compiler to perform crucial optimizations. Optimizations such as loop fusion involve rearranging program statements, which can affect the order of read and write operations on shared variables. This reordering can introduce race conditions. Without a memory model, a compiler might either avoid these optimizations in multi-threaded programs or apply them incorrectly, leading to potential bugs.

The C++ memory model defines how memory operations behave in the C++ abstract machine, specifying synchronization points like acquiring a lock or entering a synchronized block. These synchronization barriers ensure that changes to shared variables are made visible to other threads only when necessary. Race conditions are defined by the ordering of operations relative to these barriers.

By adhering to the memory model, the compiler gains flexibility in applying optimizations. It only needs to guarantee that the values of shared variables at synchronization points remain consistent between the optimized and unoptimized code. As a result, reordering statements in code blocks without synchronization barriers is considered safe.

What is the synchronized-with relation?

A synchronized with relation is only possible between operation on atomic types or operation on data structure which are implemented using atomic operations. A "suitably-tagged" atomic write operation W on a variable x synchronizes with a "suitably-tagged" atomic read operation on x if the read retrieves the value written by W or by a series of atomic read-modify-write operations where the first thread in the sequence reads the value written by W . More clearly: if thread t_1 stores a value and threads t_2 reads it, there's a synchronizes-with relationship between the store and the load that guarantees that t_2 reads what t_1 stored.

The C++ memory model allows various ordering constraints to be applied to operations on atomic types, which is what we refer to as "suitably-tagged".

What is the happens before relation?

The happens-before and strongly-happens-before relations are the basic building blocks on operation ordering in a program: they specify which operation sees the effect of which operation. In the case of a program using only one thread is simple: if one operation is sequenced before another, then it also happens-before it and strongly-happens-before it. When operations occur within the same statement, they are generally unordered, meaning no happens-before relationship is implied.

However there are two particular cases of operations on a single statement that are ordered:

- $op1 ; op2$: that is a sequence of statements separated by a comma (Grazie ar cazzo)
- $op2(op1)$: the result of $op1$ is passed as parameter of $op2$

In all other cases where multiple operations occur within a single statement, there is no guarantee of execution order.

What is the inter-thread happens-before relationship?

The inter-thread happens-before relation is relatively simple and relies on the synchronized-with relationship. If operation A on thread t_1 synchronizes with operation B on thread t_2 , then A inter-thread happens-before B . The inter-thread happens-before relation is transitive. inter-thread happens-before also combines with the sequenced-before relation: if operation A is sequenced before operation B which inter-thread happens-before operation C , then A inter-thread happens-before C . What all of this boils down to is that if you make a series of changes to data in single thread, you need only one synchronized-with relationship for the data to be visible to subsequent operation on the thread that executed C .

What are the C++ Memory Models?

There are three Memory Models in C++. They are provided by using the associated memory ordering options when using atomic operations. They are:

- **Sequentially Consistent Ordering**: Sequentially consistent ordering is the default memory model for operation ordering because it ensures that program behavior aligns with a simple, intuitive, sequential view of execution. This means operations are perceived as if they were executed in a single thread, one after another. Crucially, it prevents reordering: if one operation occurs before another in a thread, that order must be preserved and visible to all threads. In terms of synchronization, a sequentially consistent store operation synchronizes with a sequentially consistent load of the same variable. This enforces a clear ordering constraint across multiple threads. Additionally, any sequentially consistent atomic operation performed after the load must appear to occur after the store in all other threads using sequentially consistent atomic operations.

However, this constraint applies only to threads using sequentially consistent operations and does not extend to threads that use atomic operations with relaxed memory ordering.

- **Relaxed Ordering:** in the relaxed ordering there is no longer a single global order of events: different threads can see different views of the same operations. Threads do not agree anymore on the order of the events. Different CPU caches and internal buffers can hold different values for the same memory. Operations on atomic types performing relaxed ordering do not participate in synchronizes-with relations. Operations on the same variable within a single thread still obey happens-before relationships. The only requirement is that access to a single atomic variable from the same thread can't be reordered. Relaxed atomic operations are difficult to manage, they must be used in conjunction with atomic operations that feature stronger ordering semantics in order to be useful for inter-thread synchronization.
- **Acquire-Release Ordering:** in acquire-release ordering there is still no total order of operations, but it introduces the following synchronizations:
 - atomic loads are acquire operations (`memory_order_acquire`)
 - atomic store are release operations (`memory_order_release`)
 - atomic read-modify-write operations (like `fetch_add()` and `exchange()`) are either acquire, release or both (`memory_order_acq_rel`) In acquire-release ordering the synchronization is pairwise between threads that do the release of a shared variable with a thread that does the acquire of that shared variable. We still have different ordering seen by different threads, but these orderings are restricted following the synchronization between pairs.

What are the disadvantages of Sequential Consistent Ordering?

On a weakly-ordered machine with many processors, it can impose a notable performance penalty, because the overall sequence of operations must be kept consistent between processors, possibly requiring extensive and expensive synchronization operations between processors. The common x86 and x86-64 architectures (employed by AMD and Intel processors on modern PCs) don't have this problem, offering sequential consistency relatively cheaply. Still, the Sequential Consistent Ordering is the most expensive Memory Model.

What are the memory ordering options provided by atomic operations?

The memory ordering options are passed as parameters for the atomic operations. By providing a memory ordering option you enforce the Memory Model ordering associated to that ordering option on that operations. The memory ordering options are divided by the Memory Model ordering they are associated with:

- **Sequentially Consistent Ordering:**
 - `memory_order_seq_cst`: the default option if no memory ordering option is provided as parameter to an atomic operation.
- **Acquire-Release Ordering:**
 - `memory_order_consume`: it's all about data dependencies, introducing data-dependency nuances to the inter-thread happens-before relationships. FROM THE C++17 STANDARD ONWARD IT'S SAID TO NOT USE IT, SO JUST KNOW THAT EXISTS AND IGNORE IT.

- `memory_order_acquire`: can synchronize with a `memory_order_release` operation done on the same shared variable.
 - `memory_order_release`: can synchronize with a `memory_order_acquire` operation done on the same shared variable.
 - `memory_order_acq_rel`: read-modify-write operations behave as both an acquire and a release, so a prior store can synchronize with such an operation, and it can synchronize with a subsequent load.
- **Relaxed Ordering**:
 - `memory_order_relaxed`: employs a Relaxed Ordering Memory Model (all synchronizations and consistent memory order goes a PUTTANE).

What are fences?

Fences are operations that enforce memory-ordering constraints without modifying any data. They are used in conjunction with atomic operations that use `memory_order_relaxed` ordering constraints. Fences are global operations and affect the ordering of the other atomic operations in the thread that executed the fence. They are also called memory barriers for that reason. Relaxed operations on separate variables can usually be freely reordered by the compiler or the hardware. Fences restrict this freedom and introduce happens-before and synchronizes-with relationships that weren't present before.

Its possible to reorder atomic instructions following the Sequentially Consistent Ordering C++ Memory Model?

The Sequentially Consistent Ordering Memory Model implies that all threads shares a global view of the world. It means that all the operations following this Memory Model are interpreted as if are executed in sequence by a single thread even if the program is multi-threaded. For this reason operations can't be reordered: if your code has one operation before another in one thread, that ordering must be seen by all threads. Thus this Memory Model provide one ordering constraint on the operations of two or more threads. Note that this constraint doesn't carry to threads that uses atomic operations with memory ordering options different than `memory_order_seq_cst`. Those other memory ordering options are associated with either the Relaxed Ordering or Acquire-Release Ordering Memory Model, which do not enforce a single global order of events.

What theoretical computer models for Shared Memory Architectures have we seen?

A theoretical computer model is a idealized architecture that does not consider many characteristics of real computer systems (i.e. memory access time, caches, etc...). In this way when designing algorithms using a theoretical computer model we can focus only on the parallel aspects of the computation, without needing to overcome any technological limitations. We have seen the following theoretical computer models:

- **PRAM**: PRAM stands for Parallel Random Access Machine. The theoretical architecture consists of n identical processors $P_i, i \in [0, n - 1]$ operation in lock-steps. For each step a processor P_i execute an instruction cycle composed of three phases:
 - **Read Phase**: each processor can simultaneously read a single data item from a distinct shared memory cell and store it in a local register.

- **Compute Phase**: each processor can perform a fundamental operation on its local data and store the result in a register.
- **Write Phase**: each processor can simultaneously write a data item to a shared memory cell. The **Exclusive Write PRAM** variant allows writing only distinct cells. The **Concurrent Write PRAM** variant allows processors to write on the same location, which could lead to race conditions.

Three-phase PRAM instructions are executed synchronously. Communication in PRAM is implemented in terms of reading and writing to shared memory. The shared memory can be accessed in a uniform way s.t. each processor has access to any memory location in a unit of time.

There exist several PRAM variants, each defined by the different way it handles how data in shared memory can be read/written. There are four possible combinations:

- **ER**: Exclusive Read
- **CR**: Concurrent Read
- **EW**: Exclusive Write
- **CW**: Concurrent Write

The variants of the PRAM are determined by the use of two of these possible combinations. The most popular variants are:

- **Exclusive Read Exclusive Write (EREW)**: no two processors are allowed to read or write to the same shared memory cell during any cycle
- **Concurrent Read Exclusive Write (CREW)**: several processors may read data from the same shared memory cell simultaneously. Still, different processors are not allowed to write to the same memory cell.
- **Concurrent Read Concurrent Write**: Both simultaneous read and write to the same shared memory cell are allowed in this variant. In case of a simultaneous write in order to avoid a race condition we need to further specify which value will be stored. Four common approaches to deal with this situation are provided:
 - * **Priority CW**: each processor has a distinct priority and the one with the highest priority succeeds in writing its value.
 - * **Arbitrary CW**: a random processor is chosen between the ones executing the concurrent write.
 - * **Common CW**: if all the concurrent writes want to store the same value for that shared cell, then the value is written, otherwise the memory location remains unchanged.
 - * **Combining CW**: All values to be written are combined into a single value by an associative binary operation such as sum, product, minimum or logic AND.
- **BSP**: BSP stands for Bulk Synchronous Parallel. This theoretical architecture consists of three parts:
 - A collection of pairs between a Processor and its associated memory Memory (P_i, M_i) . Each Processor P_i can only see its associated Memory M_i . This model is called Distributed Memory MultiProcessor.
 - A communication network where each processor P_i can communicate with another processor P_j using point-to-point communication.
 - A Barrier constructs that synchronizes all processors in the network.

a BSP can be represented as a $BSP(p, r, g, l)$ where:

- p is the number of processors in the network
- r is the computational ratio, expressed in FLOPS
- g is the cost of communicating a data word
- l is the global synchronization cost

A BSP algorithm consists of a sequence of Super Steps. a Super Steps is a set of communications or computations done within a global barrier. These Super Steps are divided in:

- **Computational Super Step**: consists of a set of computational steps.
- **Communication Super Step**: consists of a set of communication steps.

A h-relation is a Communication Super step where each processor receive and sends at most h data words. The cost of an h-relation is: $T(h) = h \times g + l$ where:

- g is the gap, that is the time taken to send a data word.
- l is the latency, the global time necessary to synchronizes communication and enduring that all the data as been send to the correct destination

Both g and l depend on the number of processors available. The unit of time used to express this relation is the FLOP.

The cost of a Computations Super Step is: $time_of_computation(w) = w + l$ where w is the work defined as the maximum number of operations done in a Super Step by one of the Processors in the network. This means that Processors that finish early their computation have to wait for all the processors to reach the barrier.

The cost of a Mixed Super Step is : $T(h) = w + h \times g + l$

- **Work-Span Model**: This theoretical architecture abstract the parallel application into several components s.t. each is a sequential portion of the application. Then we represent the application as a graph s.t. each node is a component (i.e. sequential portion) and the arcs describe dependencies between them. Given two components A and B s.t. $A \rightarrow B$ is a dependency read as: A produces some data that is needed by B . In the Work Span model we define two metrics:

- **Work**: we define as work the total amount of time spent executing the application.
- **Span**: is the total amount of work needed to complete the longest chain of components from the input component (those with no input dependencies) and the output component (those that do not need to satisfy other component dependencies). The "longest" chain is the chain of components from input to output that requires the longest time to compute. The time spenti compute the Span is declared as t_∞

To execute the application in parallel we need to spend at least the time for the Span metric. The time to compute the whole application with just one processing element is the Work, denoted as t_1 . For the Worker Span Model the maximum speedup is:

$$\lim_{n \rightarrow \infty} speedup(n) = \frac{t_1}{t_n} = \frac{t_1}{t_\infty}$$

In the Work Span Model the super linear speedup is not possible. In the Work Span Mode we have the lower bound and upper bound of the speedup determined by the Bent's Lemma.

- **Data-Flow and Macro Data-Flow**: is a general approach to parallelism based on data dependencies among the programs operations. It is interpreted as a **Graph** where each **node** is an **instruction** and **edges** represent **data dependencies** (i.e. read-after-write dependencies) among those instructions. Macro Data -flow is a **Data-Flow** where instead of interpreting the **nodes** as single instructions, they are represented as **entire sequential functions or block of code**.

Difference regarding speedup between the Work Span Model and the Amdahl's Law

The **Work Span Model** is more pessimistic than **Amdahl's Law**, since it further limits possibilities for parallel execution beyond serial fraction.

What are FLOPS?

FLOPS stands for **Floating Point Operations Per Second**, and it is a **measure of a computer's performance**. It quantifies **how many floating-point arithmetic operations** (such as addition, subtraction, multiplication, and division of real numbers) **a system can perform in one second**.

What is the Brent's Lemma?

The Brent's Lemma gives us **an upper bound and a lower bound on the speedup we can achieve**:

- **Upper Bound**: having P as the number of processing elements: $T_p \leq \frac{(T_1 - T_\infty)}{p} + T_\infty$
- **Lower Bound**: having P as the number of processing elements: $T_p \geq \frac{T_1}{p}, T_p \geq T_\infty$

What is the Shared Memory System?

A Shared Memory System is a **type of parallel computer architecture where all CPUs can access a common memory space through a shared bus or a crossbar switch**. In addition to the shared Main Memory **each core has access to a smaller Local Memory** (i.e. the L1 Cache) in order to reduce expensive access to the main memory. In order **to guarantee correctness, value store in local caches must be coherent with the values stored in shared memory**. This is called the **cache coherence protocol** and it is supported by all modern multi-core systems often referred as non-uniform access architectures (NUMA).

Parallelism in a shared memory systems is typically created by starting threads running concurrently on the system. Exchange of data is implemented by threads reading from and writing to shared memory locations. Thus multiple threads work on the **same data** simultaneously, and programmers need to implement the **required coordination among threads wisely**.

What problems can arise from Shared Memory Applications? (Race Condition Definition)

Shared Memory Architecture can be subjected to **race conditions** **when two or more threads access a shared variable simultaneously. The final stored value can not be determined deterministically because operations can interfere with each other, leading to unpredictable or incorrect results.** This typically happens when **one thread modifies data while another reads or writes to it at the same time.** **Preventing race conditions requires mechanisms like locks or atomic operations to control access to shared resources.**

What is the Distributed Memory System?

Distributed Memory Architecture is a type of parallel computer architecture where all CPUs are connected through an interconnection network. Each CPU can only operate on data stored in its local memory. Remote data accesses need to be explicitly implemented through message passing of the network.

The interconnection network is an important architectural factor of distributed memory systems. It is typically implemented with point-to-point links or a switching network. Standard Network Protocols (i.e. Ethernet) are often used to implement the communication tasks. The network topology determines the scalability of the architecture for many applications. Prominent examples are compute clusters and network-on-chip architectures.

What problems can arise from Distributed Memory Systems?(Data Partitioning)

Since all CPUs have a separated memory space we have Data Partitioning, that is the distribution of data between processes. The challenge is to partition the data efficiently so that each node can process its portion with minimal communication overhead. Poor partitioning can lead to uneven workload distribution, causing some nodes to be overloaded while others are underutilized. Additionally, frequent data exchanges between nodes can significantly degrade performance, as communication between nodes is much slower than accessing local memory.

Show to me the differences between Shared Memory System and Distributed Memory System

Shared Memory Systems and Distributed Memory Systems are both parallel computer architectures. In Shared Memory Systems the Parallel Resources can operate on both a global Shared Memory and a Local Memory (i.e. L1 Cache). In Distributed Memory Systems the CPU can access only their local memory, if they want some information stored in another Resource they need to explicitly communicate with it. Shared Memory Systems uses a single process where multiple threads work concurrently while Distributed Memory Systems uses multiple processes that communicate between each other. The initialization overhead between a thread and a process can be more than two orders of magnitude, making threads more lightweight to manage. Shared Memory Systems are vulnerable to Race Conditions, while Distributed Memory Systems are vulnerable to Data Partitioning.

Levels of parallelism on a single CPU

In order to improve their compute performance modern microprocessors incorporate several levels of parallelism. Flynn's Taxonomy identifies four classes of architectures according to the instructions and data streams:

- **SISD (Single Instruction, Single Data):** refers to the tradition Von Neumann architecture where a single sequential processing element works on a single stream of data.
- **SIMD (Single Instruction, Multiple Data):** performs the same operation on multiple data simultaneously
- **MIMD (Multiple Instruction, Multiple Data):** uses multiple Processing Elements to execute different instructions on different data streams.
- **MISD (Multiple Instruction, Single Data):** employs multiple Processing Elements to execute multiple instructions on a single stream of data. This type of parallelism is not so common but can be found in pipelined architectures such as systolic arrays

Additionally modern CPUs and GPUs contain a number of features to exploit different level of parallelism:

- **Multiple Cores:** Modern microprocessors use MIMD parallelism by incorporating a number of cores that can execute threads asynchronously and independently
- **Vector Units:** Modern architectures exploit data-level parallelism by incorporating SIMD based vector units within each core. Vector units can execute vector instructions on a number of data items simultaneously.
- **Instruction Level Parallelism:** current processors further exploit instruction-level parallelism (ISP) by pipelining and superscalar execution of instructions. Pipelining overlaps the execution of the different stages of multiple instruction such as instruction fetch, decode, register fetch, execute memory access and register write-back.
in superscalar parallelism multiple execution units are used to execute multiple instructions simultaneously. In order to take advantage of superscalar parallelism, execution instructions might be reordered by the system through out-of-order execution (OOE).
- **SIMD Concept:** SIMD architectures exploit data parallelism by issuing the same instructions to all available Processing Elements or ALUs in each clock cycle. They require a centralized control unit in order to dispatch instructions. This simplifies the design of Processing Elements since they do not need any additional logic for program control. Frequent usage of conditional statements can significantly reduce the performance of SIMD systems. Today's microprocessors include SIMD parallelism at a smaller scale.
- **Vectorization on Common Microprocessors:** one way to exploit the compute power of the available vector registers is through the use of intrinsics. Intrinsics are assembly coded function and data type definitions that can be used in C and C++ source code.

What are AOS and SOA in SIMD Parallelism?

In order to exploit the power of SIMD parallelism it is often necessary to modify the layout of the employed data structures. There are two ways to store a sequence record where each record consists of a fixed number of elements:

- **Array of Structures (AOS):** simply stores the records consecutively in a single array.
- **Structure of Arrays (SOA):** uses one array per dimension. Each array only stores the values of the associated element dimension.

What is Model Driver Rewriting in Skeleton Frameworks?

Consider a skeleton program Δ , the target architecture, the implementation of the skeleton Δ and the performance model associated. The problem of finding the best rewriting of Δ becomes quite difficult.

Suppose we have a set of rewriting rules $\mathcal{R} = \Delta_1 \rightarrow \Delta_2, \Delta_3 \rightarrow \Delta_4$ then we can generate the rewriting tree $RwTr(\Delta, \mathcal{R})$:

- Δ is the root of the tree
- if a rule $\Delta_i \rightarrow \Delta_j$ with $i \neq j$ applies to node $N_k \in RwTr(\Delta, \mathcal{R})$ where N_k is a skeleton expression matching Δ_i then N_k is linked to a node where the occurrences of Δ_i have all been substituted by Δ_j

Programming Language Abstractions in Model Driver Rewriting?

In a structured parallel programming framework, parallel applications are developed using programming abstractions thought constructs of the programming language used. Frameworks implemented using C++ (like FastFlow) provides classes modelling parallel patterns (like the Farm). The task of the programmer consists in picking up the more suitable pattern or pattern compositions among those available in the framework. Note that the same parallel application may be implemented using different pattern compositions.

How to define Function Equivalence in Model Driven Rewriting?

Different refactoring rules determine function equivalence among different patterns. These rules can tell us how different pattern implementations leads to the same parallel application.

Show me the Map Fusion Rule

Suppose we have two map patterns in sequence that work on the same Collection of input items. The first map applies function f to each element in the collection. After the first map finishes the second map applies function g on the collection returned by the first map. This process can be rewritten as a single map pattern that applies to the collection the composition of f with g . Formally:

$$map(f) \circ map(g) \equiv map(f \circ g)$$

Show me the Pipe Rule

$$pipe(f, g) \equiv f \circ g$$

Passing from the left implementation to the right, the efficiency is improved as the grain of the computations to be execution in parallels increases. As a consequence the communication and synchronization overheads are better tolerated

Show me the Farm rewriting rule

We denote with Δ Skeletons or Skeletons compositions. Then $\forall \Delta_i. farm(\Delta_i) \equiv \Delta_i$. This is because the farm skeleton does only impact the parallelism degree used to compute the program, not its functional semantics. For the same reason:

$$pipe(\Delta_1, \Delta_2) \equiv pipe(farm(\Delta_1), farm(\Delta_2)) \equiv farm(pipe(\Delta_1, \Delta_2))$$

What is a Skeleton Normal Form?

When considering the Skeleton Rewriting System, we may be interested in defining the "better" rewriting with respect to some non-functional feature. This better rewriting is called "Normal Form" to represent that this is the program representation that would be normally considered for the implementation. The Normal Forms are the following:

- $T_s(pipe(\Delta_1, \Delta_2)) = max\{T_s(\Delta_1), T_s(\Delta_2)\}$
- $T_s(farm(\Delta)) = max\{T_{Emitter}, T_{Collector}, \frac{T_{Worker}}{n_w}\}$
- $T_s(comp(\Delta_1, \Delta_2)) = L(\Delta_1) + L(\Delta_2)$
- $T_s(seq(C)) = L(C)$

Where $L(x)$ is the latency of x and $T_{Emitter}$, $T_{Collector}$ and T_{Worker} being the Service Time of the Emitter, Collector and Worker respectively.

Determining the Normal Form

The set of "leaves" in the Skeleton Tree Δ is computed from left-to-right. This set is called **fringe** and represents the **set of sequential wrapping Skeletons appearing in the Skelton Program**:

- $\text{fringe}(\text{seq}(C_i)) \equiv C_i$
- $\text{fringe}(\text{comp}(C_1, \dots, C_n)) \equiv \{C_1, \dots, C_n\}$
- $\text{fringe}(\text{pipe}(\Delta_1, \Delta_2)) \equiv \text{append}(\text{fringe}(\Delta_1), \text{fringe}(\Delta_2))$ where *append* append the second parameters at the end of the first.
- $\text{fringe}(\text{farm}(\Delta)) \equiv \text{fringe}(\Delta)$

The Normal Form guarantees the best Service Time from a Stream Parallel Skeleton Composition by reducing communication overhead.

Tell me why $\text{map}(f) \circ \text{map}(g) \equiv \text{map}(f \circ g)$ and $\text{pipe}(\text{map}(f), \text{map}(g)) \equiv \text{map}(f \circ g)$. How much resources we employ in each case?

- $\text{map}(f) \circ \text{map}(g) \equiv \text{map}(f \circ g)$: this is valid by the Map Fusion Rewriting Rule which states that $\text{map}(f) \circ \text{map}(g) \equiv \text{map}(f \circ g)$.

The left-hand expression is slower than the right-hand expression since we need to spread the workers in 2 maps.

- $\text{pipe}(\text{map}(f), \text{map}(g)) \equiv \text{map}(f \circ g)$: this is valid by the normal form of the pipe which states that $\text{pipe}(\Delta_1, \Delta_2) \equiv \text{append}(\Delta_1, \Delta_2)$.

The difference is in the throughput: the left-hand expression has $\text{throughput} = \max(T_{\Delta_1}, T_{\Delta_2})$. In the right-hand expression has $\text{throughput} = T_{\Delta_1} + T_{\Delta_2}$.

In which condition I can implement the Vectorization? If I have a library call inside a for and I use the vectorization (think of a *pragma*) what happens? And if I have dependencies and I force the vectorization?

I have to follow this **guidelines** in case of a vectorization in a loop:

- I need to **know the loop count statically**. The loop count must stay **constant** for all its execution.
- The **end** of the loop must depend only to the **counter in its guard**
- **No Switch** statements, the **if statements** can be done in **parallel**.
- **No function calls** inside the loops are **allowed except inline functions or intrinsic functions**.
- **No data dependencies** (i.e read-after-write like $A[i] = A[i - 1]$). In case there are data dependency the vectorization is cancelled and the code is executed sequentially.
- **use `_restrict`** for pointers in order to specify that there is no aliasing between pointers.

What is a Behavioural Skeleton?

A Behavioural Skeleton **support automatic management of non-functional features related to the skeleton implementation** (for example the performance). The **Separation of concerns between the pattern implementation and non-functional features** allows:

- **very effective adaptation policies to be implemented.**
- **very clean and effective interfaces to be implemented.**
- **allows management policy reuses across different skeletons with similar implementation.**

What is the structure of a Behavioural Skeleton?

- **Parallel Pattern**: the parallel pattern implementation template. The passive part provides the interface needed to operate the pattern. It handle the task computation and retrieve the final result.
- **Autonomic Manager**: the manager taking care of non-functional features (read the following question for more information)
- **Autonomic Controller**: embodies the interfaces needed to operate on the parallel pattern execution, both the monitoring/triggering and actuator ones.

What are Autonomic Managers?

An Autonomic Manager is an entity employed in Behavioural Skeletons that takes care of non-functional features of the pattern implementation. It is composed of a passive part and an active part. The Autonomic Manager passive part provides instead the non functional interfaces, that is those interfaces that may be used from outside to monitor behavioural skeleton execution (that is to retrieve measures and parameters relative to the behavioural skeleton execution progress) and/or to intervene on the behavioural skeleton manager policies (that is to re-define the management policies of the manager). The Execution Schema of the Autonomic Manager is divided in two steps:

- **Decision Step**: the behavior of the application in execution is monitored and analyzed. Decisions are taken concerning the adaptation of the application if it does not perform as expected. The phase is logically divided in two sub-phases:
 - **Monitoring**: the monitoring activity is planned at compile time by inserting proper monitor "probes" (i.e. clocks to measure the time spent for specific parts of the application) in the application. The monitoring phase can be handled by the compiling tools or the user. In general the monitoring phase is impacted by the kind of probes available on the target architecture.
 - **Analyze**: after determining the current application behavior in the Monitoring Phase, the Analyze fare takes up decision duty. We need to have some kind of abstract performance modelling which provide us with a comparison between performance figure computed by the model and those obtained during the Monitoring Phase. In case the two measures do not match, adaptation is required. Possible abstract actions are:
 - * increase/decrease of the parallelism degree
 - * complete restructuring of the parallel application like changing the parallelism exploitation pattern or the pattern composition used to implement the pattern.
- **Execution Step**: the plan determined by the Decision Step is devise and eventually executed in order to implement them. The phase is logically divided in two sub-phases:
 - **Plan**: During this phase we expose the parallel structure of the application (i.e. we take a look at the skeleton tree). In this way we identify the precise points of intervention on the running application. The solution decided is required to impact the least possible the running application. At the end of the plan phase we have an ordered list of actions.
 - **Execute**: the phase simply schedules and apply the actions determined by the Plan Phase on the running application. The actions are executed from left-to-right.

Note: all of the above steps are executed while the application is running.

What is FastFlow?

FastFlow is a framework for parallelizing in a Shared Memory System similar to OpenMP. It is implemented as a header-only library.

What are the two level interface of FastFlow?

The interface of FastFlow is composed of two layers:

- **Building Blocks:** the lower layer which is `closed to C++`. This layer offers a set of blocks to build network of threads.
- **Parallel Patterns:** the upper layer offers some implementations that use network implementations done with Building Blocks.

What is the FastFlow Programming Model?

A FastFlow Application is a `directed graph whose nodes are computing entities and edges are channels carrying data references` (i.e. pointers). The entities are either sequential or parallel building blocks. The `nodes can be stateless or stateful and they synchronize through message passing for accessing shared data`. The consistency of the data is ensured by the Runtime System. **When a node send a pointer, that node forgets it.**

How is handled Channel Communication in FastFlow?

A channel connects two nodes using a `Single Produce Single Consumer FIFO queue`:

- **Input Non-Determinism:** suppose you have a single output node and three input node. Each input node communicate with the output node using each their own channel. The receiver non-deterministically receive data from one of the channels.
- **Output Selectivity:** suppose you have a single input node and three output nodes. The input node is connected through each output node using an unique channel. The sender can decide with which output channel communication each time. You can also let the RunTime decide non-deterministically where to send the data.

Input Non-Determinism and Output Selectivity are possible because the communication between nodes is implemented using distinct channels made with independent Single Produce Single Consume FIFO queues. We do not use concurrent buffers to avoid using atomics and locks.

How is Concurrency Control handled in FastFlow?

By default we have `non-blocking concurrency control`. If a receive has to receive something, it never blocks. It does a busy waiting loop requesting the data it needs actively. We can explicitly decide to use a blocking behavior which is implemented in FastFlow using standard mutexes and condition variables.

To make the behavior of your application blocking you must compile your code with the flag `-DBLOCKING_MODE`. It is also possible to switch to blocking mode at run time by using `optimize_static` or the `blocking_mode()` member function provided by all Building Blocks.

0.0.1 When to use Blocking and Non Blocking?

If the number of threads used in your application is greater than the number of cores, the busy-waiting loop of the non-blocking paradigm could impact the application performance. Thus in that case it is better to switch to a blocking approach. As an example consider a Farm where the Collector node, that only sporadically receives data elements, that shares the same core as a Worker or even worse the Emitter.

What is the Channel Capacity in FastFlow?

By default the channel buffers in FastFlow are unbounded. That means that if the producer produces lots of data and the consumer is not able to keep up, the application blows memory. It is possible to bound a buffer by specifying a fixed predetermined size. **The only exception to this rule are the Feedback Channel, they can be only unbounded.** The reason is that since Feedback Channels are defined as channels that return data from the output node to the input node (backward), they make a cycle with the normal channel that connects input to output node. If the Feedback Channel is bounded it could cause deadlocks.

What are Eso-Streams and Endo-Streams in FastFlow?

In FastFlow a stream is a sequence of data values, possibly unbounded, coming from one or more sources. The stream items coming from the same source usually have the same data types. Streams are categorized in two groups:

- **Eso-Stream:** coming from a real source like a network interface, a file, a sensor, ...
- **Endo-Stream** is generated directly by the application, mainly unrolling the iterations of a loop

FastFlow was born for streaming data computations, but also can handle data parallel computation.

Do we have source nodes and sink nodes in FastFlow?

We don't have the concept of source node or sink node in FastFlow. The first node of the network in FastFlow is indicated as the source node, but in reality it is not different than any other node in the Graph. For the same reason the last node in the network is called the sink node, but it is semantically equivalent to any other node. Still, it is possible to build by using Building Blocks a more specialized streaming framework where we have a source node and a sink node.

What role have the source and sink node in a network (NOT IN FASTFLOW)?

The source node ingests data into the system, acting as the entry point for data streams, it continuously reads data from external sources like databases, APIs, or files. The sink node is the endpoint that consumes and stores or processes the output data, it outputs data to destinations like databases, message queues, or storage systems. Source nodes initiate the flow, while sink nodes finalize it.

What is a Sequential Node in FastFlow?

A node has associated two queues:

- **input queue:** where it can receive pointers
- **output queue:** where it can send pointers

Even the source node of a network in FastFlow has an input queue, it will simply be ignored. To build a FastFlow node we have to create a C++ class that **must** derive from `ff_node_t<IN_t,OUT_t>` where:

- `IN_t`: specify the type of the stream elements the node receive from the input queue.
- `OUT_t`: specify the type of the stream elements the node sends to the output queue

It is possible to overload the following methods:

- `OUT_t* svc(IN_t* in)`: we specify how the node uses the input data to produce the output data. **This is the only method that is mandatory to define when defining a new node.** The RunTime of FastFlow will call the `svc` method when there is an item in the input queue. The item returned by the method will be putted in the output queue by the FastFlow Runtime. The method can return four types of messages:
 - A memory pointer to a data item it want to send to another node.
 - `EOS`: stands for End of Service, tells the node in the output to terminate its execution. Terminate also the execution of the node returning it.
 - `GO_ON`: tells to the receiving node to remain alive.
 - `ff_send_out`: produces more than one output.
- `int svc_init()`: the method is called only once at the initialization of the node.
- `void svc_end()`: this method is called before terminating the node or when the node is put to sleep. **Note that it returns nothing, it is not possible to send anything into the output queue.**
- `void eosnotify(size_t ch)`: a node terminate when received the special messaged `EOS`. The first stage (source node?) can send the `EOS` message and then the message will propagate terminating all nodes. This method is called each time an `EOS` message is received by the node. It is mainly used when you have FeedBack Channels

How are handled Multi-Input and Multi-Output Sequential Node

- **Single-Input Multi-Output Node**: this node has one input buffer and multiple output buffers. To define a node like this you make a class derived from `ff_monode_t<IN_t, OUT_t>`. In this class we have two extra methods:
 - `ff_send_out(ptrdata)`: send non-deterministically the output message to one of its output buffers
 - `ff_send_out_to(ptr_data, channelID)`: send the data through a specific channel.
- **Multi-Input Single-Output Node**: in this node there are many input channels and one output channel. It is created by defining a class derived from `ff_minode_t<IN_t, OUT_t>`. There are two extra members:
 - `ff_get_channel_id()`: to know from which channel the node received the input.
 - `fromInput()`: returns true if the input comes from one of the input channels. It is used when there are also Feedback Channels to consider.

There is a Multi-Input Multi-Output node in FastFlow?

By choice there isn't a Multi-Input Multi-Output node. Still, it can be created manually combining a Multi-Input Single-Output node with a Single-Input Multi-Output node.

How works the FastFlow RunTime System?

The FastFlow Runtime System spawns a thread for each node created. So if there are 100 nodes in a network, then the runtime will spawn 100 threads.

Each thread executes the following pseudocode:

Algorithm 3 Pseudocode describing the FastFlow RunTime for each node

```
if svc_init() < 0 then
    break;
end if
in = input_channel.pop();
if in == EOS then
    eosnotify();
    output_channel.push(EOS);
    break;
end if
out = svc(in);
if out == GO_ON then
    continue;
end if
output_channel.push(out);
while out != EOS do
    svc_end();
end while
while !shouldrestart() do
    nothing;
end while
```

How to start a node?

The start of the thread associated with the corresponding node is done through the `run()` method. To stop the thread there is the `wait()` method. **Be aware that this are private methods, so they cannot be called by the user.** To start execution of a node the user calls the `run_and_wait_end (bool = false)` public method. **If the returned value by the method is negative, then an error occurred and the user must throw an exception.**

Combining Sequential Nodes

Sequential nodes can be combined using the `ff_comb` Building Block. To create a combined node you pass as parameters to the method the nodes you want to combine. The method will return a new node obtained by the combination of the previous which runs in a single thread. We can combine also already combined nodes. **Parallel BuildingBlocks cannot be combined!**

What are the FastFlow Parallel Building Blocks

- **Pipeline:** nodes we have written can define a pipeline. Pipelines can be combined together to define another pipeline.
- **Farm:** there is a sequential node called Emitter which communicate with a pool of nodes called Workers. All the Workers derive from the same node.
- **All-To-All:** is a generalization of the farm. Instead of having only an Emitter node we have a list of nodes called L-Workers where each one is an Emitter. They are connected

to one or more nodes called R-Workers that have the role of Workers. This is preferred to a normal farm when the Emitter can cause a potential bottleneck. It is possible in a All-To-All to nest every Building Block you want, provided that it is embedded in a pipeline

Use the Building Blocks of FastFlow as Lego blocks, you connect them to obtain the desired network.

What is the difference between `ff_Farm` and `ff_0farm`?

- `ff_Farm(Workers, Emitter, Collector)`: this generates an unordered farm which preserves with some restrictions the input ordering.
- `ff_0farm()`: generates an empty ordered farm.

What are the composition rules for Building Blocks?

- two sequential building blocks can be connected in pipeline regardless of their input/output cardinality.
- a parallel Building Block can be connected to a sequential Building Block through a multi-output and multi-input node.
- two parallel Building Blocks can be connected wither if they have the same number of standard nodes or through multi-input/multi-output nodes.

Adding a Feedback Channel

Feedback Channels have their data flow in the opposite direction respected the standard data flow. **Be aware that Feedback Channels can cause deadlocks!** They are used to route back results, or to provide control information to the previous node. To define that a Building Block has Feedback Channels use the `wrap_around` public method.

How is termination handled with Feedback Channels?

In a Building Block with Feedback Channels it could happen that even though the previous node in the network sends EOS to the next node, the latter one does not get it immediately and instead get a data from its Feedback Channel. To overcome this problem is using the `eosnotify()` method overridden with the defined condition of termination.

What is the Farm and All-To-All Task Scheduling policies?

there are two scheduling policies:

- **Round Robin**: The default policy for the scheduling of Workers by the Emitter is Round Robin (actually is loose Round Robin “wink wink”). Tasks (data pointers) are assigned to Workers starting from 0,1,2, The Round Robin works only if the capacity of the channels is unbounded. If the buffer capacity is bounded, then if the Emitter tries to send a message to Worker i but it fails due to its buffer capacity being full, then the Emitter will skip Worker i and try to send the message to Worker $i + 1$. This scheduling is called Loose Round Robin.
- **On-Demand**: here the Workers when free ask the Emitter to give them a new task. The implementation of the on-demand scheduler is not 100% accurate to the auto-scheduling policy, rather it closely emulates it. It is implemented by forcing the Workers to have a bounded capacity for handling just one pointer. This means that the Emitter keeps trying to send a message to each Worker in the queue, until it finds the first free Worker.

What is CPU Affinity?

CPU Affinity is the feature to **control the mapping of threads to cores of the running architecture**. Deciding the optimal mapping threads-cores is a NP-Hard problem. However there are several **heuristic that are implemented by OSs that works well**. If the platform and the application structure are known in advance, then it is possible to derive good mapping strategies. For example in Linux the `taskset --c` command allows user to bind the process execution to a given set of cores.

CPU Affinity in FastFlow

The syntax of CPU Affinity in FastFlow is not so flexible. At the time of writing the mapping from threads to cores is specified by a string defined in the `config.hpp` header file and is called `FF_MAPPING_STRING`. Within this string you specify the core IDS separated by `,`.

During the spawning of a thread the FastFlow Runtime look into the `FF_MAPPING_STRING`, pick the first ID and assign it to the apposite thread. It tries to map threads to cores close to each other.

Suppose you have a Producer/Consumer pattern, it would be convenient to map a Produce thread and a Consumer thread in near cores s.t. when they communicate we can exploit some level of cache. If the `FF_MAPPING_STRING` is empty, the RunTime tries to pin threads to cores, but assumes a linear numbering of cores. The `-d NO_DEFAULT_MAPPING` compilation flag disables the default thread mapping.

Is it possible to map threads at runtime?

It is:

- in the `svc_init` method by calling the `ff_mapThreadToCpu` function.
- by using the `optimize_static` function or the `no_mapping()` method of `Far`, `A2A` and `Pipeline Building Blocks`.

Additionally there is the `mapping_string.sh` script which uses a tool called `hwlock` to compute a new string of cores for your machine.

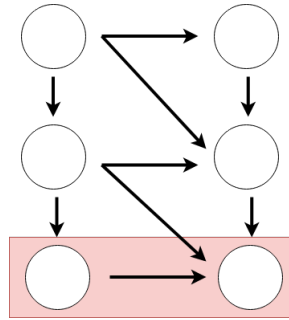
When to use thread mapping

In application when you have many threads the rule of thumb is to disable thread mapping. Instead in application with fine-grained computations, the best option is to control the affinity.

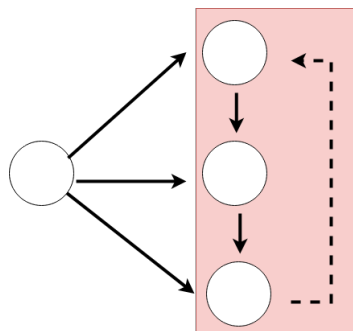
Main difference between FastFlow and OpenMP's CPU Affinity

In FastFlow since we know the Topology a priori, it is possible to configure the CPU Affinity manually.

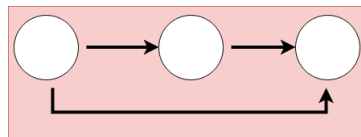
What Graphs cannot be made in FastFlow and why?



- This graph could be made using an All-To-All if not for the last node of the L-Workers. In an All-To-All that node would be a Multi-Output node, which the node on the graph isn't.



- This graph could be made using a Farm with the exception that Workers aren't able to communicate with each other



- This graph could be made using a Pipeline with the exception that a node cannot communicate with a node from an afar stage.

Macro Data-Flow in Fast Flow

In FastFlow Macro Data-Flow is implemented by the `ff_mdf` High Level Pattern. To express data dependencies we use a specialized node called Task Generator. The Macro Data-Flow is then implemented as two pipeline stages:

- **First Stage:** generate the data according to the dependencies of the program
- **Second Stage:** A Farm with Feedback Channels called Engine which execute ready instruction. A ready instruction is an instruction whose data dependencies have been satisfied.

How to construct a Macro Data-Flow in FastFlow

When constructing the `ff_mdf` pattern the user have to explicitly declare `INPUT` and `OUTPUT` data dependencies for each macro task and provide pointer to input and output data. To create a macro task we use the `AddTask` method. The task queue is managed by the Emitter of the Farm.

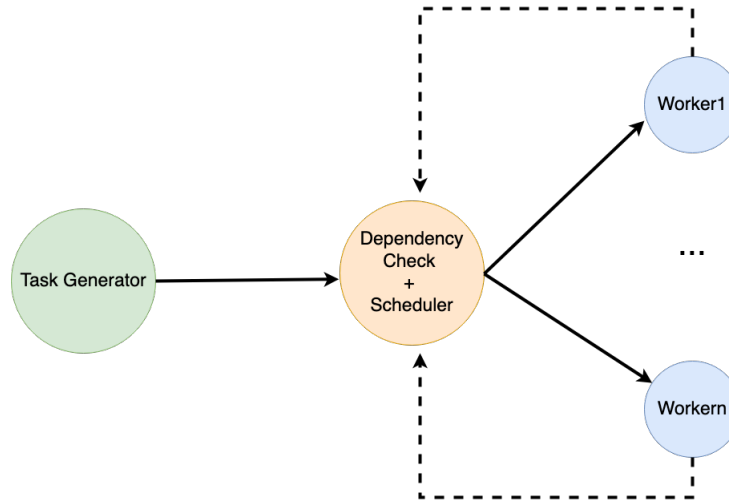


Figure 4: Structure of a Macro Data-Flow in FastFlow

What are the High Level Patterns in FastFlow?

In FastFlow an High Level Pattern is built atop of Building Blocks. We have seen three High Level Pattern:

- **Parallel For:** it mimic the Parallel For of OMP. It is implemented using a Farm with a special Emitter that schedules the iterations to the Workers. Each Worker has a feedback channel with the Emitter, returning the result of their computations. The Emitter applies a reduction function to the returned values. There is also a second version called `ParallelForPipedReduce` which delegates the reduction function to a Collector called `Reducer`. The chunk size tells how many iterations at a given time you want a single Worker to execute. There are three possible cases:
 - **chunk size = 0:** static scheduling. Each Worker gets a contiguous chunk of iterations (i.e. $\frac{\#iterations}{\#workers}$)
 - **chunk size > 0:** dynamic scheduling with task granularity equal to chunk size
 - **chunk size < 0:** static scheduling with task granularity equal to chunk size. Chunks are assigned to Workers in a Round Robin fashion.
- **Divide and Conquer:** It is present with the name `ff_DC`. It is implemented using a Master-Worker skeleton. Each Worker executes the algorithm in parallel. Data dependencies are managed by the farm's Emitter. The Emitter takes care of the macro instructions. To define a Divide and Conquer computation you need:
 - A divide function that splits the data
 - a combiner function that combines the data
 - a function that checks if we are at the base case
 - a function that checks if the condition for recursion is satisfied
 - the construct patter `ff_DC`. It is used by the Workers

- **Macro Data-Flow:** See the question above
- **Map:** wraps a ParallelForReduce pattern into a single-input single-output node. Inside a pipeline or a farm it is generally better to use the FastFlow map than a ParallelForReduce since the map removes the Emitter and adds other small optimizations like the mapping of worker threads, scheduler disabled and ...

Combining Data and Stream Parallelism in FastFlow

Having a Parallel-For implementation permits us to combine data-parallelism and stream-parallelism together. For example we can construct a pipeline containing a Farm where its Workers use a ParallelFor. Its important to pay attention to the affinity, disabling the affinity because you will have the threads spawned by the parallel for and the threads spawned by the Farm and Pipeline.

What are the Collectives of MPI?

MPI offers an assortment of asymmetric communications patterns in the form of Collectives. Each Collective can be implemented **manually** by the user using point-to-point communication, but Collective offer a **better implementation**. These construct are called Collective from the fact that all processes, independent from their role, needs to call the same function. We have seen:

- **Barrier:** the barrier is a construct which make a **process wait until all the other processes reach the barrier**.
- **Broadcast:** the Broadcast operation **sends from a source a given buffer to all other processes in the Communicator. Recall that all processes need to call Broadcast**
- **Reduce:** the reduce is the **complimentary of Broadcast. Here we have a number of processes send the data to one target**. The received data is then applied to a reduction operation s.t. at the end the node receives a single data item.
- **Scatter:** **a source process sends to all processes a portion of a particular block of data.** Be aware that for the Scatter to work the number of total elements should be divisible to the number of processes.
- **Gather:** **is the complimentary of Scatter. We collect data from the processes and return all of them to a single process.**
- **Allgather:** is equivalent to a **Gather + Broadcast**. The set of data is received by all nodes.

How can we obtain the time execution in a MPI application?

MPI provides its own function called **MPI_Wtime()**. Call it at the beginning and end of a computation.

How to end the execution of a MPI application

The Master process should receive from all the Workers a EOS message. To recognize the EOS message the Worker must send it with a **eos_tag**. The **eos_tag** you decide it by assigning it an integer (use an enum to better recognize it). When the processes have all finished then call the **MPI_FINALIZE** function to end the application.

In what system MPI is used?

Is used in clusters or supercomputer machines that consists of several compute nodes interconnected through a network. Each node contains its own memory. We are in a distributed memory systems and the defacto standard to handle this kinds of application is the Message Passing Interface (MPI). Note that MPI is only a definition of an interface, different implementation are provided by different application. In our case we have used the OpenMPI C++ framework.

SPMD Style

MPI follows the SPMD (Single Program Multiple Data) style: it split the workload into different tasks that are executed on multiple processors. By following this style a MPI program consists of several processes with their own local memory. Communication among processes is handled through the interconnection network by using send and receive routines. The majority of current clusters contain shared-memory nodes that are interconnected through a network forming a hybrid distributed-memory/shared-memory system. Still, MPI is able to spawn several processes on the same machine.

Number of processes during the execution in MPI

The number of processes remains constant during the whole execution. Processes are mapped to different nodes. The user can specify in which node each process needs to be assigned in a configuration file. If any process fails, the whole application crashes.

How the communication is handled in MPI?

Most of MPI applications depend on communication between processes suing ping-pong communication. The source and destination processes must indicate in their code that they are part of the communication:

- **Send Procedure:** the send procedure in the source process involves storing the data into a buffer and notifying the communication device that communication has started through the `Send`.
- **Receive Procedure:** the destination process must acknowledge with a receive routine that it wants to receive the data. Once this happened, the data can be transmitted and both processes may continue their work. The function to do this is `Recv`.

Here we are considering a blocking communication. Still, MPI offers the functions `Isend` and `Irecv` for non blocking communication between processes. The only substantial difference is that they return `MPI Request` objects. a `MPI Request` object offers the following methods:

- `Wait()`: block the computation until the message have been sent/received.
- `Test()`: returns a boolean that indicates if the operation has finished. Note that this method never blocks the process calling it

How create new datatypes in MPI?

Its possible to use the predefined data provided by MPI as building blocks for allowing programmers to define their own derived types. There are three kinds of derived datatypes:

- `MPI::Datatype Create_contiguous(int count)`: creates a new datatype defined as count contiguous elements of another existing type.

- `MPI::Datatype Create_vector(mucho_texto)`: it can be used to reference equally-spaced, fixed-sized blocks of data. Each block is simply a concatenation of `block-length` elements of an old datatype and the spacing `stride` between blocks is multiple of the extent of the base datatype. The number of blocks in the datatype is specified by `count`.
- `MPI::Datatype Create_struct(mucho_texto)`: this is the more general datatype. It allows for the replication of a base datatype in non-contiguous blocks. However, block sizes and strides can vary.

What is a Communicator in MPI?

A Communicator is an object that represents a set of processes that can communicate between each other by using Collectives operations. The default communicator is the universal `MPI::COMM_WORLD` which contains all the processes in the running application.

To define a new communicator MPI provides the routine `Split(int color, int key)`. The function returns a new communicator that is a subset of the previous one. `color` identifies the subset where the process is included and `key` indicates the rank of the process within the communicator.

What are the `MPI::COMM_SIZE` and `MPI::COMM_RANK` routines?

- `MPI::COMM_SIZE`: returns the number of MPI processes in the application.
- `MPI::COMM_RANK`: returns the rank of the process. The rank is the unique ID of the process and $\in [0, \text{MPI::COMM_SIZE} - 1]$

What is SLURM?

SLURM is a `workload manager`. In big compute clusters it's important to have a resource management system that handles the nodes/CPU/GPU or any kind of compute resource. Almost all big clusters use SLURM. SLURM does the following:

- Resource allocation by assigning resources to users
- Scheduling jobs. A job in SLURM is something that needs at least a resource. Decides when to start giving the requested resources and relinquish them at the end of the job execution.
- Distribute the total workload by balancing the work over machine resources.
- Monitoring and accounting.

What is the structure of a cluster?

In a cluster we have:

- a number of MASTER nodes used as login nodes (in the case of the SPM Cluster it's the frontend node).
- Internal nodes used for computations.

Partitions

SLURM divide computational nodes into **partitions** (also called queues). Partitions are then configured in different ways:

- **limiting the number of hours per job**
- **limiting the number of nodes per job**

In the SPM Cluster we have only **one partition with heterogeneous compute nodes**. To know which partitions are available on a cluster use the command **sinfo**.

What are the SLURM policies?

SLURM assign the available nodes of a partition according to some level of priority. How to determine the priority depends on the configuration. In general SLURM assign the available nodes of a partition to the submissions with highest priority. The priority depends on the policy used:

- **Backfill Algorithm**: the default implementation attempts to schedule low-priority jobs if they do not prevent higher-priority jobs from starting. Slowdowns can occur if the users overestimate the time for their jobs.
- **Priority Queue Algorithm**: it uses a global queue ordered according to a certain priority assigned to each job. The queue is FIFO. As a downside SLURM could block itself if the job at the head of the queue cannot be executed because the cluster don't have enough resources.

What is the SPM Cluster configuration?

One login node and **8 compute nodes connected through 10 Gigabit Ethernet**. One SLURM queue (**only one partition**). This partition employs a **Backfill policy**. When you reserve one job to node you get the whole node.

Interconnection Networks

There are two main network types:

- **Shared**: it can communicate **one message at a time** (i.e. a bus in a Ethernet connection).
- **Switched**: are able to **simultaneously transfer several messages between different pairs of nodes**.

In a high-performance distributed memory switched networks are preferred

Network Topology

Network Topologies determines the **scalability and performance of a parallel computer architecture**. A network is interpreted as a connected graph whose nodes are switches/processors and edges represents communication link. They classified as:

- **Direct Networks**: all nodes have a processor attached, e.e. there are direct connection between processors.
- **Indirect Networks**: there could also be intermediate routing-only nodes.

There are three **features** used to compare the qualities of different network topologies:

- **Degree**: is the maximum number of neighbors of any node.
- **Diameter**: the length of the longest of all shortest path between any two nodes.
- **Bisection-Width**: the minimum number of edges (or links) to be removed to disconnect the network into two halves of equal size. If the number is odd one of the two networks can have an extra node.

We define the following **desiderable properties**:

- **Constant Degree**: the degree of a network should be independent of the network size. In this way we can scale a network to a large number of nodes without needing an excessive number of connections.
- **Low Diameter**: to support efficient communication between pair of processors, the diameter should be minimized.
- **High Bisection-Width**: bisection-width identifies a potential bottleneck of a network and has an implication on its internal bandwidth. A low bisection-width can slow down many collective communication operations and thus can severely limit the performance of applications. Achieving an high bisection width may require non-constant network degree.

What **topologies** have we seen?

- **Linear Array**: a topology with n nodes $P_0, \dots, P_{n-1}$ denoted as L_n . $\forall i \in (0, n-1)$. P_i is connected with P_{i-1} and P_{i+1} and thus the degree of the network is $\deg(L_n) = 2$. The longest distance between any two nodes is between the leftmost node P_0 and the rightmost P_{n-1} . Data communicated between them needs to traverse $n-1$ links and thus $\text{diam}(L_n) = n-1$. The bisection-width is 1 since only the link between $P_{\frac{n-1}{2}}$ and $P_{\frac{n}{2}}$ needs to be removed to split L_n into two disconnected halves.
- **2D Mesh**: a 2D mesh is a linear array with its dimensionality increased by 1. The n nodes are arranged in a grid of size $n = k \times k$, denoted as $M_{k,k}$. For simplicity assume k is always even. It can be seen that a node can at most be connected to four other nodes, thus the degree $\deg(M_{k,k}) = 4$. Note that the degree is independent of the actual mesh size. The longest distance is between the upper leftmost node and the bottom rightmost node or the opposite. This requires traversing $2 \times (k-1)$ edges, thus $\text{diam}(M_{k,k}) = 2 \times (k-1) = 2 \times (\sqrt{n} - 1)$. In order to split $M_{k,k}$ we need to split it by the middle row or column, removing at least k edges. Thus $\text{bw}(M_{k,k}) = k = \sqrt{n}$.
- **2D Torus**: an extension of the 2D Mesh. 2D torus $T_{k,k}$ extends $M_{k,k}$ by adding wrap-around edges to directly connect the left node of each row and the right node of each row as well the bottom node of each column with the top node of each column. This reduces the diameter and bisection width by a factor of 2 compared to the 2D mesh of the same size while keeping the degree constant at 4.
- **3D Mesh**: a 3D Mesh $M_{k,k,k}$ extends a 2D Mesh by another dimension.
- **3D Torus**: extension of the 3D mesh. It has a constant degree, relatively low diameter and relatively high bisection-width. For this reason it has been used as an interconnection network in a number of Top500 supercomputers.
- **Binary Tree**: we can represent an interconnection network as a binary tree of height and depth d denoted as BT_d . The $n = 2^d - 1$ nodes are arranged in a complete binary tree of depth d . Each node that isn't a root or a leaf is connected to its parent and its children, thus $\deg(BT_d) = 3$. The longest distance in a BT_d occurs when travelling between a leaf

node on the left half of the tree to one on the right half or vice versa. It requires going up to the root and then down again, thus $\text{diam}(BT_d) = 2 \times (d - 1) = 2 \times \log_2(n + 1)$. Note that the degree is constant and the diameter is low. The bisection width $\text{bw}(BT_d) = 1$: just remove the link of the root and we have split the network in two halves.

- **Fat Tree (Fiderio)**: it address the extremely low bisection width by introducing more links nearer the top of the hierarchy.
- **HyperCube**: Denoted as $Q_d (d \geq 1)$ is a graph with $n = 2^d$ nodes that has each vertex labeled with a distinct bit string of length d . Two vertices are connected if and only if their bit-string differ in exactly one bit. These means that each node is connected to exactly d other i.e. $\text{deg}(Q_d) = d = \log_2(n)$. The longest distance between two nodes occurs when the two associated bit-string differ in every position. In this case d bitflips are required in order to transform one bit string into the other thus $\text{diam}(Q_d) = d = \log_2(n)$. Removing all links between the nodes starting with label 0 and all nodes starting with label 1 disconnects in two equal halves. It holds that bisection width is $\text{bw}(Q_d) = 2^{d-1} = \frac{n}{2}$. Among all the networks we have see the hypercube has the highest bisection width. Its disadvantage is the non-constant degree, making it difficult to scale up to large number of nodes. For this reason hypercube topologies have been used in some early message-passing machine, but nowadays are mostly forgotten.
- **Dragonfly**: The network is divided into groups of routers, where each group has routers fully interconnected. Routers within a group are connected in a all-to-all fashion, minimizing latency inside the group. Routers in different groups are connected using a few direct links between them, reducing the need for long paths. This topology allows for scalable, low-latency communication by reducing the number of hops needed for long-distance data transfers, making it efficient for large-scale systems. Degree: Typically $p - 1 + k$ where p is the number of routers in a group and k is the number of inter-group links. Bisection Width: High, proportional to inter-group links. Diameter: Typically 3 hops.

Foster's Parallel Algorithm Design Methodology

Assume we have a problem or sequential program to parallelize. There isn't a clear recipe to do so, having several different possible solutions. In order to explore possible parallel solutions there is the parallel algorithm design methodology developed by Foster. It is subdivided in four stages:

- **Partitioning**: we identify potential sources of parallelism by partitioning the problem into small tasks. There are two different partitioning strategy:
 - **Domain Decomposition**: we first determine an appropriate partitioning scheme for the data and then deal with the computation.
 - **Functional Decomposition**: we first decompose the computation and then deal with the associated data. Different partitioning strategies leads to different solutions. We want to identify as much parallelism as possible, aiming at maximizing the number of identified tasks. Note that data parallelism leads to finer granularity compared tot ask parallelism.
- **Communication**: we determine the required communication between the tasks identified in Stage 1. We specify the data that must be transferred between two tasks in terms as a channel linking htese tasks. On a channel one task can send a message and the other can receive. In practice there are many communication patterns (local/global, structured/unstructured, static/dynamic, synchronous/asynchronous).

- **Agglomeration:** we want to increase the granularity of our design from Stage1 to Stage2 by combining a number of small tasks into larger tasks. The reason behind this stage is that executing a large number of small tasks in parallel using different processes can be highly inefficient due to communication overhead. To overcome this we agglomerate several small tasks into a single larger task. This often improves data locality, reducing the communication overhead.
- **Mapping:** we map tasks to processors for execution. The mapping procedure aims at:
 - minimizing communication between processors by assigning tasks with frequent interactions to the same processor.
 - enabling concurrency by assigning tasks to different processors that can execute in parallel.
 - balance the workload between processors.

Differences between multiprocessing and multithreading

- **Multiprocessing:** parallelize a program over multiple compute units in order to exploit redundant resources such as registers and ALUS of different CPU cores to speedup computations.
- **Multithreading:** shares the hardware resources such as caches and ram of a single core or multiple cores in order to avoid idling of unused resources.

Whats is the ABA problem and possible workarounds?

In multithreaded computing, the ABA problem occurs during synchronization, when a location is read twice, has the same value for both reads, and the read value being the same twice is used to conclude that nothing has happened in the interim; however, another thread can execute between the two reads and change the value, do other work, then change the value back, thus fooling the first thread into thinking nothing has changed even though the second thread did work that violates that assumption. Below is a sequence of events that illustrates the ABA problem:

- Process P_1 reads value A from some shared memory location
- P_1 is preempted, allowing process P_2 to run
- P_2 writes value B to the shared memory location
- P_2 writes value A to the shared memory location
- P_2 is preempted, allowing process P_1 to run
- P_1 reads value A from the shared memory location
- P_1 determines that the shared memory value has not changed and continues.

Although P_1 can continue executing, it is possible that the behavior will not be correct due to the "hidden" modification in shared memory.

A common case of the ABA problem is encountered when implementing a lock-free data structure. If an item is removed from the list, deleted, and then a new item is allocated and added to the list, it is common for the allocated object to be at the same location as the deleted object due to MRU memory allocation. A pointer to the new item is thus often equal to a pointer to the old item, causing an ABA problem.

Possible Workarounds to the problem are:

- **Tagged state reference:** A common workaround is to add extra "tag" or "stamp" bits to the quantity being considered. For example, an algorithm using compare and swap on a pointer might use the low bits of the address to indicate how many times the pointer has been successfully modified. Because of this, the next compare-and-swap will fail, even if the addresses are the same, because the tag bits will not match. This is sometimes called ABA since the second A is made slightly different from the first. Such tagged state references are also used in transactional memory. Although a tagged pointer can be used for implementation, a separate tag field is preferred if double-width CAS is available. If "tag" field wraps around, guarantees against ABA do not stand anymore. However, it has been observed that on currently existing CPUs, and using 60-bit tags, no wraparound is possible as long as the program lifetime (that is, without restarting the program) is limited to 10 years; in addition, it was argued that for practical purposes it is usually sufficient to have 40-48 bits of tag to guarantee against wrapping around. As modern CPUs (in particular, all modern x64 CPUs) tend to support 128-bit CAS operations, this can allow firm guarantees against ABA.
- **Intermediate nodes:** A correct but expensive approach is to use intermediate nodes that are not data elements and thus assure invariants as elements are inserted and removed.
- **Deferred Reclamation:** Another approach is to defer reclamation of removed data elements. One way to defer reclamation is to run the algorithm in an environment featuring an automatic garbage collector; a problem here however is that if the GC is not lock-free, then the overall system is not lock-free, even though the data structure itself is. Another way to defer reclamation is to use one or more hazard pointers, which are pointers to locations that otherwise cannot appear in the list. Each hazard pointer represents an intermediate state of an in-progress change; the presence of the pointer assures further synchronization [Doug Lea]. Hazard pointers are lock-free, but can only track at most a fixed number of elements per thread as being in-use. Yet another way to defer reclamation is to use read-copy update (RCU), which involves enclosing the update in an RCU read-side critical section and then waiting for an RCU grace period before freeing any removed data elements. Using RCU in this way guarantees that any data element removed cannot reappear until all currently executing operations have completed. RCU is lock-free, but isn't suitable for all workloads.